



Métodos de Pesquisa e Ordenação em Estruturas de Dados

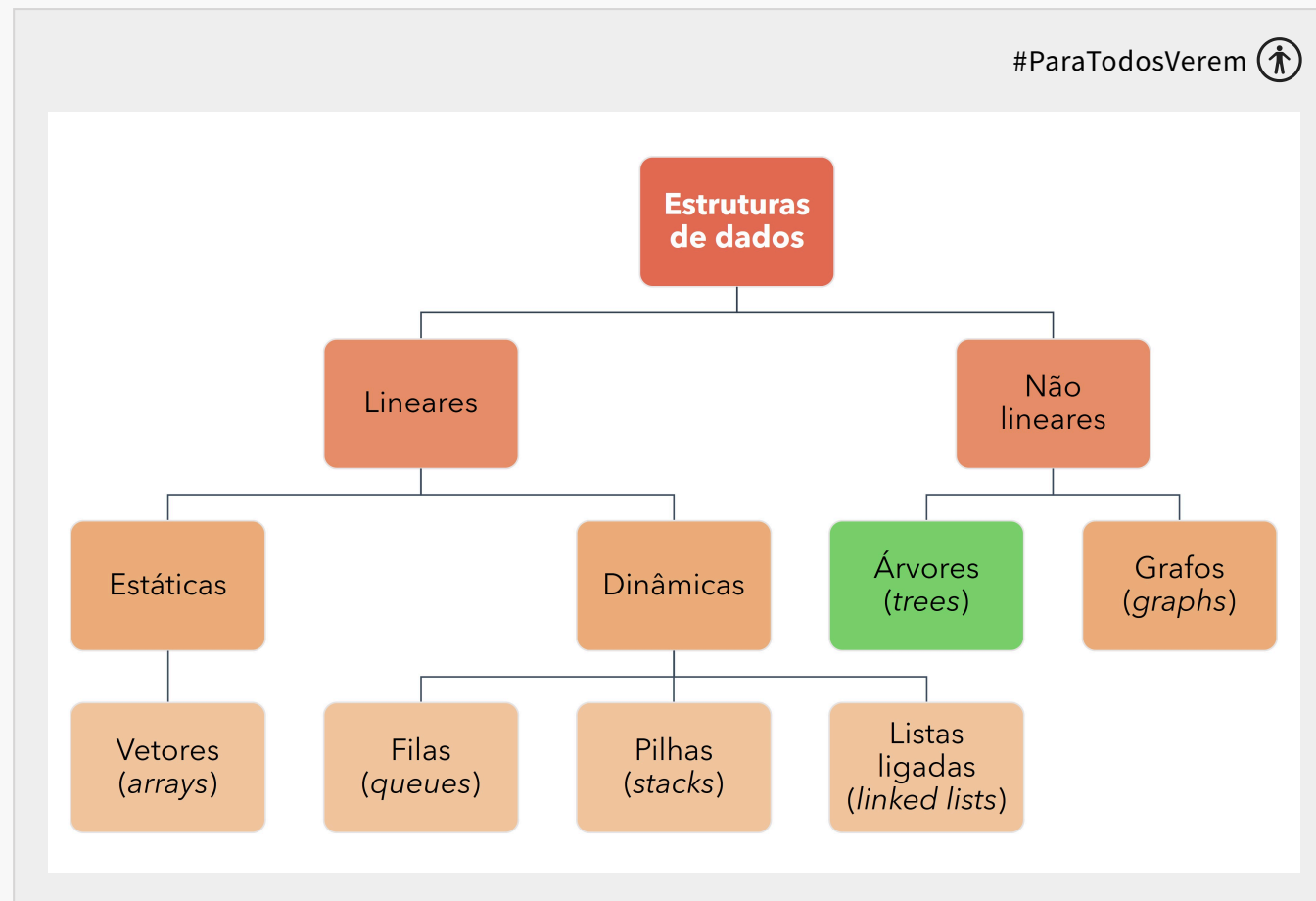
UNIDADE 05

Estrutura de dados: Árvores

Olá!

Nesta semana, fecharemos os estudos sobre estruturas de dados ao falarmos sobre **árvores**. Então, até o momento, veremos os principais tipos de estruturas de dados: vetores, filas, pilhas, listas ligadas, grafos e árvores. Entender as diferenças no comportamento e na implementação dessas estruturas de dados permitirá a você para que escolha a melhor estrutura para desenvolver o seu *software*. Além disso, permitirá que você entenda melhor a lógica de alguns algoritmos de busca e ordenação.

Vamos lá?

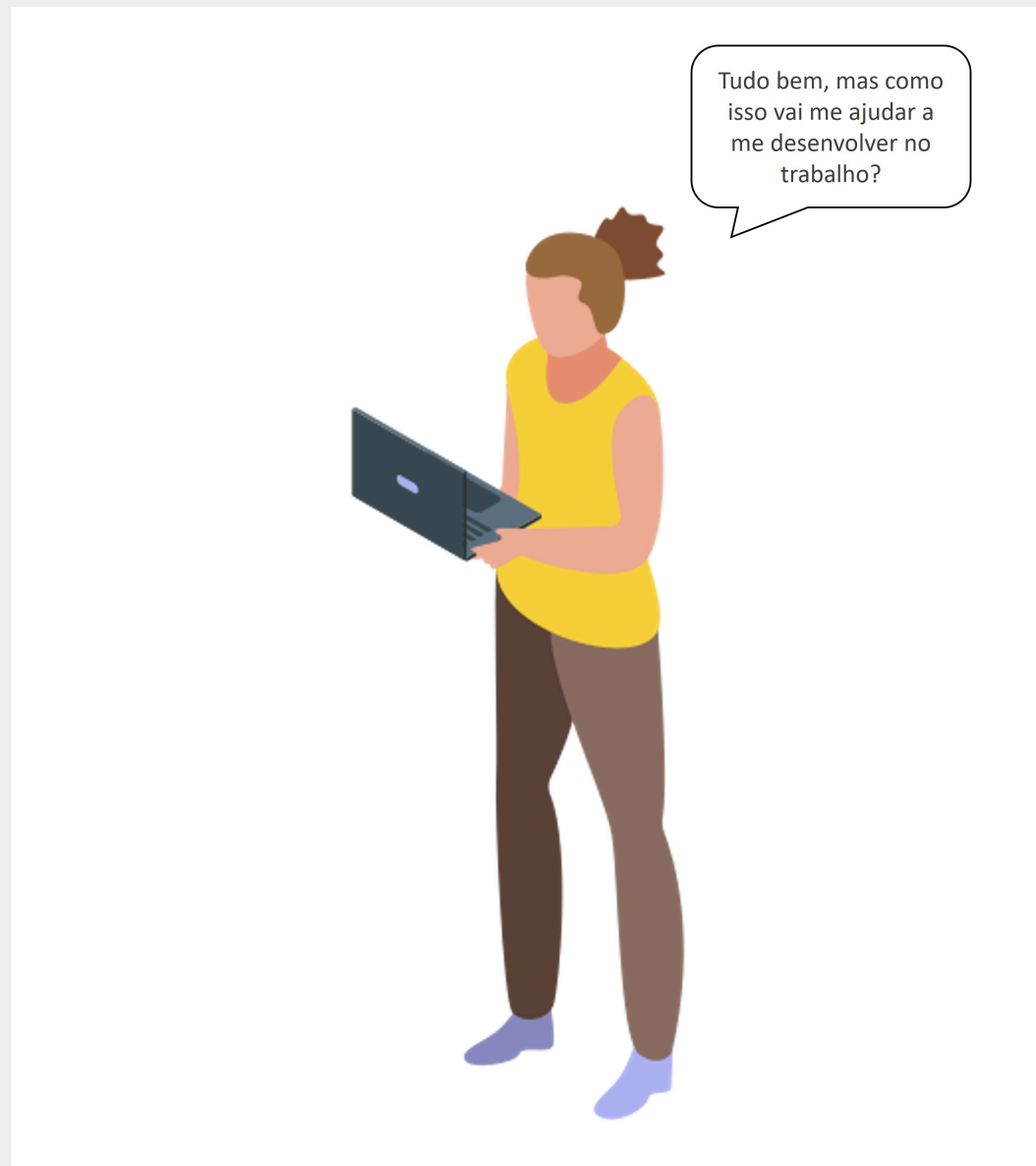


Fonte: O autor (2023).

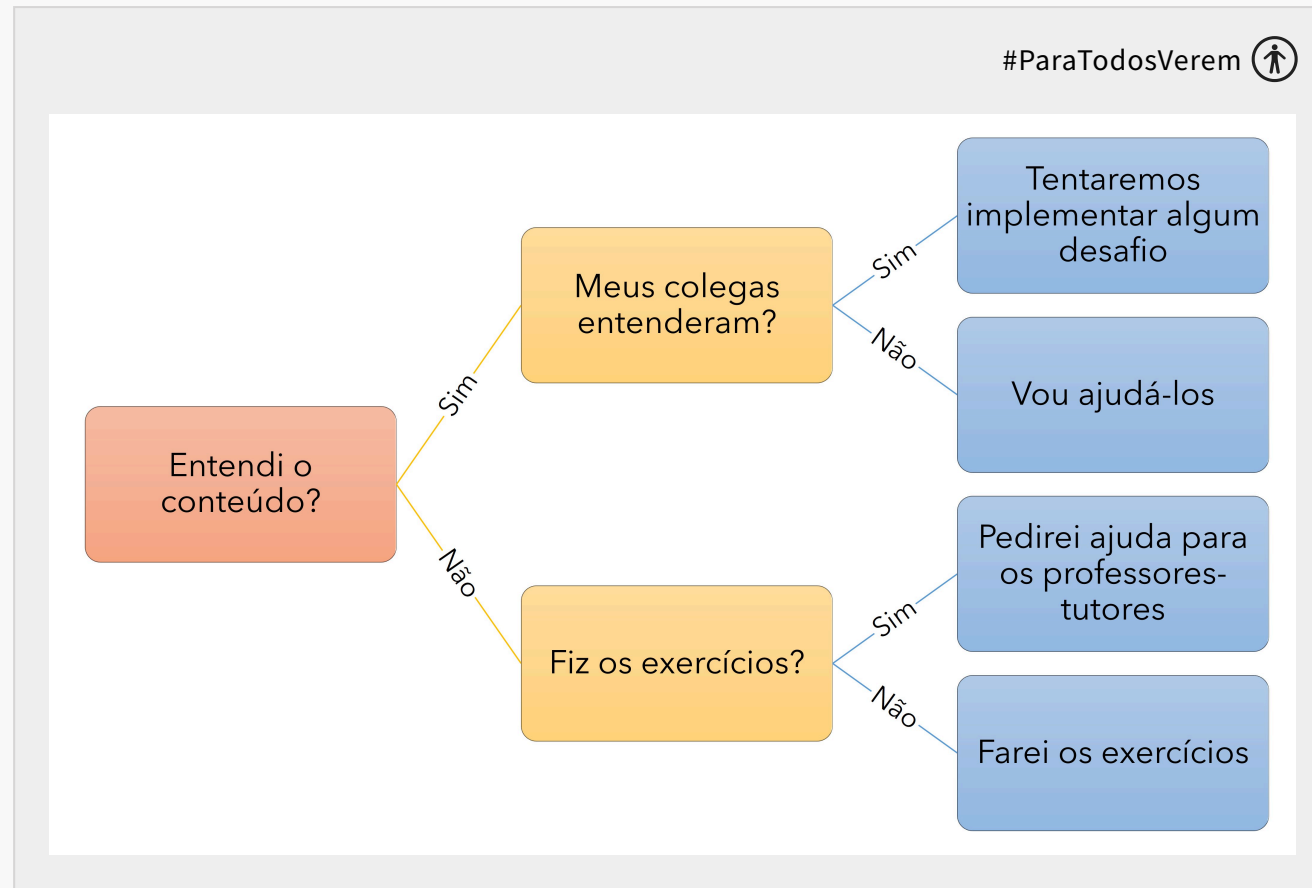
Se alguma vez você já se perguntou como os bancos de dados conseguem recuperar informações tão rapidamente, ou como um sistema de recomendação parece “saber” o que você quer antes mesmo de você terminar de digitar, está prestes a descobrir um dos segredos por trás desses feitos: as árvores em estruturas de dados.

Aprender sobre árvores vai além de simplesmente passar por esta disciplina: para que você possa se aprofundar em áreas mais avançadas da computação, como algoritmos, bancos de dados e até mesmo inteligência artificial, precisará obrigatoriamente aprender sobre árvores. São as árvores nos ajudam a organizar, buscar e manipular dados de maneira eficiente, permitindo-nos resolver problemas complexos de maneira mais intuitiva e otimizada.

Agora, você deve estar se perguntando:



No mercado de trabalho, as árvores estão em toda parte! Pense na maneira como um mecanismo de busca retorna resultados relevantes em questão de milissegundos, ou em como um jogo de *videogame* decide qual movimento o adversário deve fazer a seguir. E não para por aí: aplicações de *e-commerce* que filtram produtos, sistemas de GPS que calculam a rota mais rápida e *software* de gerenciamento de projetos que determinam dependências de tarefas – todos estes, de alguma forma, aproveitam o poder das árvores para entregar soluções eficazes. Veja o exemplo a seguir: é uma árvore de decisão sobre o que faríamos dentro da disciplina.



Um exemplo de árvore de decisão. Fonte: O autor (2023).

Então, enquanto embarcamos nesta jornada de exploração das árvores, quero que você se lembre de uma coisa: o que estamos prestes a aprender não é apenas teoria. São ferramentas práticas, usadas todos os dias, para moldar a tecnologia e o mundo ao nosso redor.

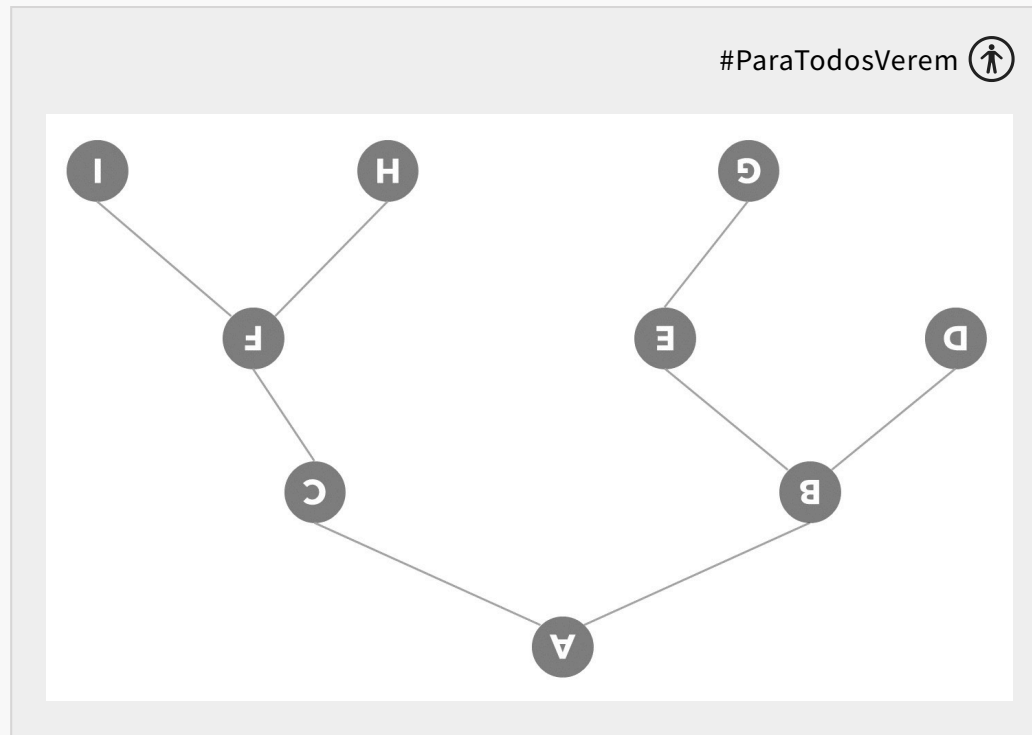
| Estrutura de Dados: Árvore

Quando falamos em árvores em ciência da computação, ainda que não estejamos nos referindo às árvores da natureza (aquelas que são verdes e que ficam plantadas na terra), usamos muitos dos conceitos delas. Veja só: assim como as árvores biológicas têm raízes, troncos e galhos, as árvores em estruturas de dados têm a sua própria hierarquia e organização.

Uma árvore é uma coleção de elementos, chamados **nós**. Mas, ao contrário de listas e vetores, os nós em uma árvore têm uma **relação hierárquica**. Começamos sempre por um nó especial, chamado de **raiz**. Este é o nó inicial, e todos os demais nós surgem a partir dele.

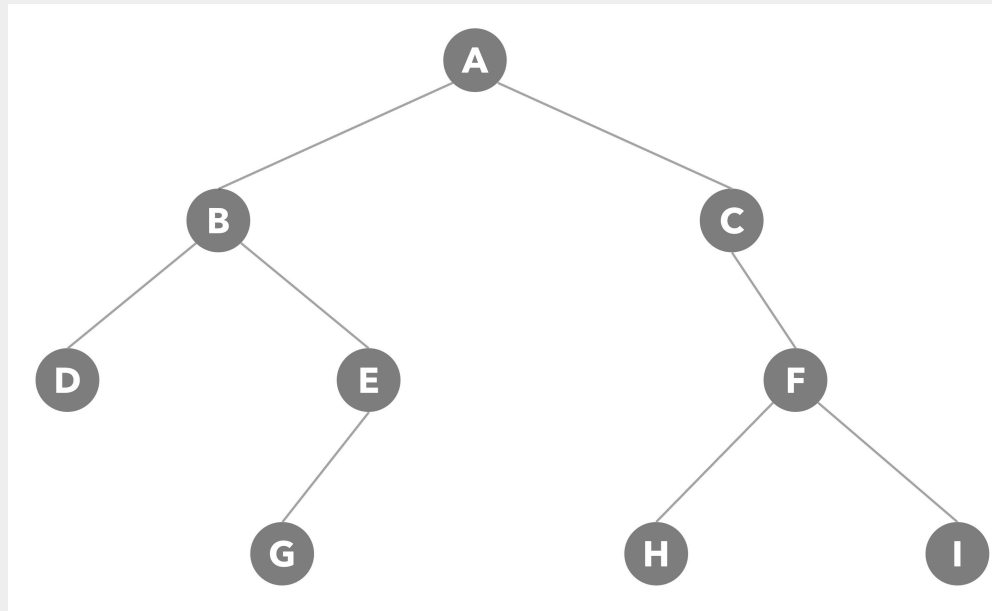
A partir da raiz, os nós podem ter um ou mais **nós filhos** e, curiosamente, qualquer nó da árvore (exceto a raiz) tem exatamente apenas um nó pai. Estes nós, que não têm filhos, são chamados de **folhas**. Veja o exemplo a seguir de uma árvore binária (isto é: todo nó pode ter 0, 1 ou 2 nós filhos). Eu sei que ela está de cabeça para baixo. Veja o formato dela: ela não se parece com algum tipo de planta que cresce a partir da base e,

conforme vamos olhando para cima, mais galhos nascem em diferentes direções? Pois bem: esta é a ideia da árvore.



Estrutura de dados: árvore binária de cabeça para baixo (sim, não é um bug: foi de propósito). Fonte: O autor (2023).

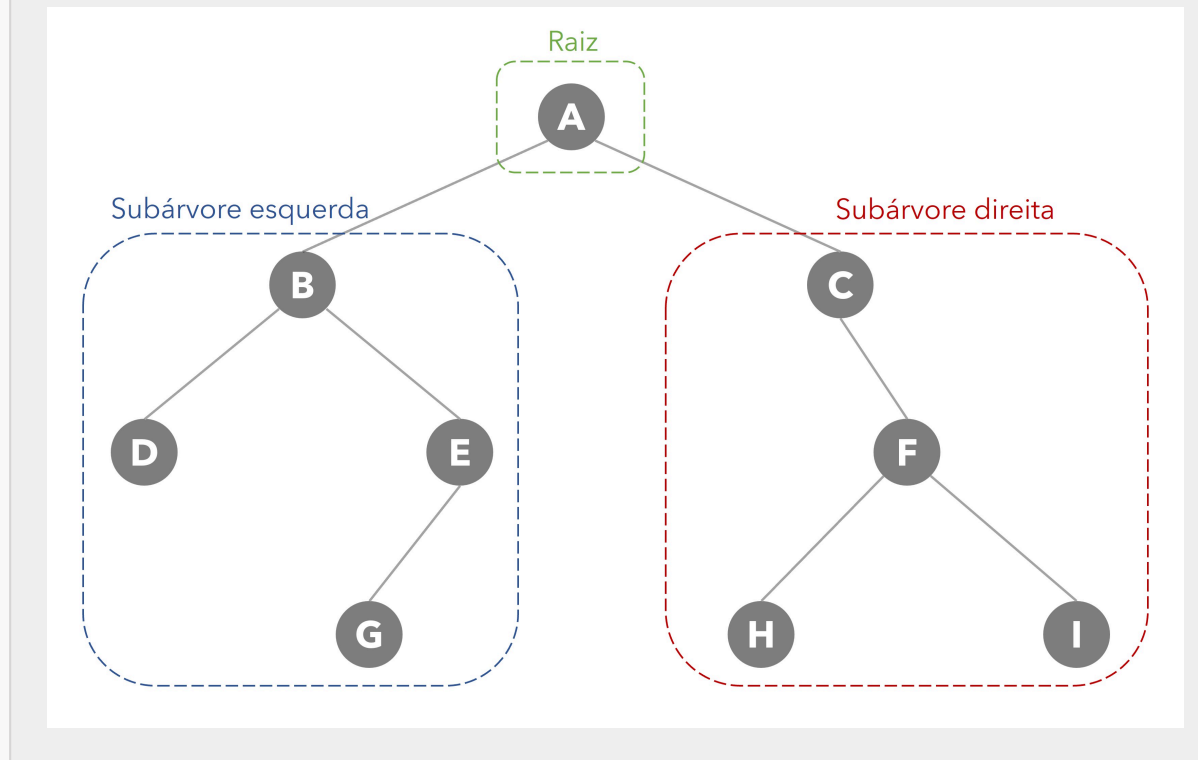
Já em TI é comum lermos a árvore de **cima para baixo**: afinal, sempre lemos os documentos e figuras começando pela parte superior, não é? Por isso, o normal é representarmos as árvores assim:



Estrutura de dados: árvore binária. Fonte: O autor (2023).

O que acha de identificarmos os elementos nesta árvore que acabamos de ver? Existem três componentes em uma árvore binária: a **raiz**, uma **subárvore esquerda** e uma **subárvore direita**, como podemos ver na figura a seguir. Cada subárvore também é uma árvore binária e pode ou não estar vazia. Além disso, as **subárvores esquerda** e **direita** são conjuntos disjuntos de nós, e independentes um do outro.

Estrutura de dados: árvore binária e os seus componentes.



Estrutura de dados: árvore binária e os seus componentes. Fonte: O autor (2023).

O elemento da raiz de uma subárvore é chamado nó **pai**. Um determinado nó que não possui filhos (subárvores esquerda e direita são vazias) é chamado de nó **folha**. Assim, de acordo com a figura apresentada, temos:

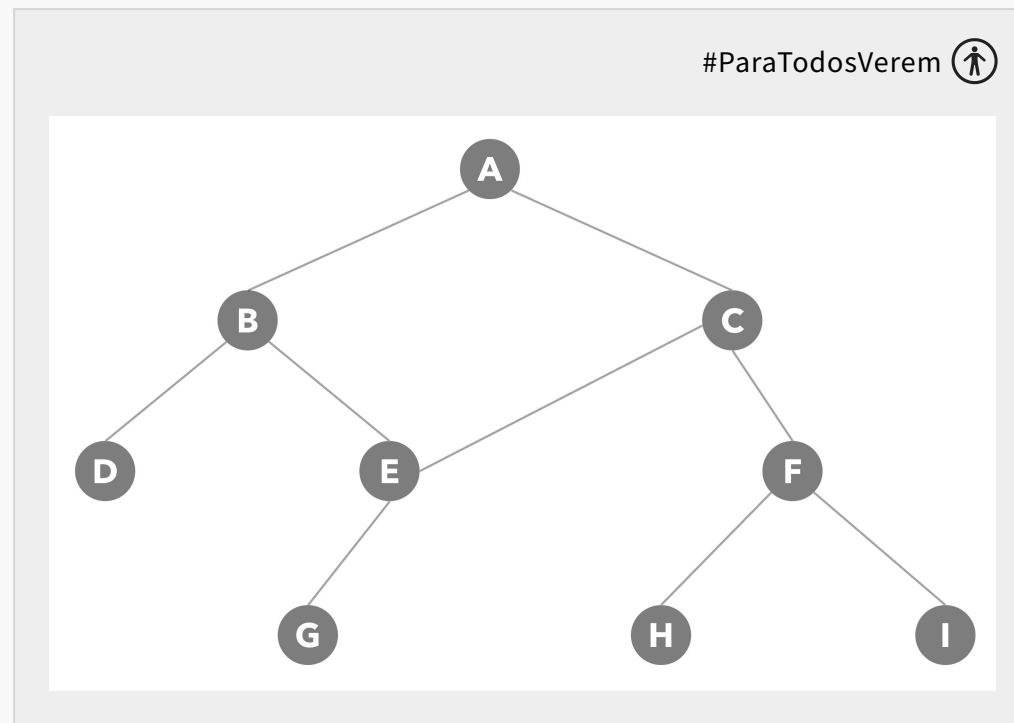
- A é a **raiz**;
- Pais:
 1. A é o **pai** de B e C;
 2. B é o **pai** de D e E;
 3. E é o **pai** de G;
 4. C é o **pai** de F;
 5. F é o **pai** de H e I.
- Filhos:
 1. B e C são **filhos** de A;

2. D e E são **filhos** de B;
3. G é **filho** de E;
4. F é **filho** de C;
5. H e I são **filhos** de F.

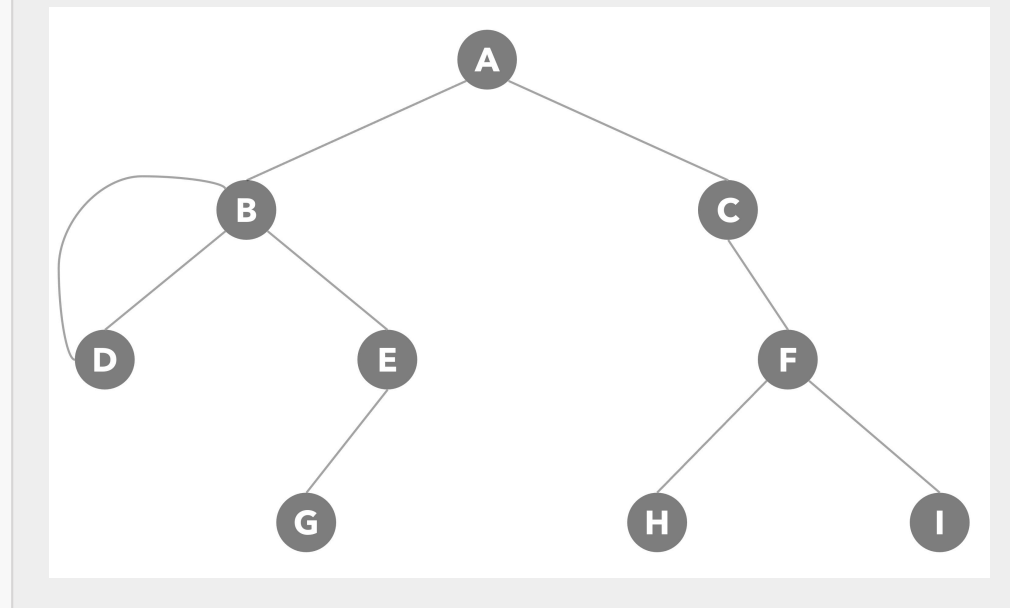
- Folhas:

1. Os nós D, G, H e I são nós **folhas**.

Relembrando: a árvore que acabamos de ver é **binária**, o tipo mais comum de árvore que temos em computação. Ou seja: um nó pode ter estritamente até dois filhos, e um filho possui somente um pai. De fato, este será o tipo de árvore que nos aprofundaremos a seguir. Dito isso, veja dois exemplos a seguir de árvores não binárias:



Árvore não binária – Exemplo 1. Fonte: O autor (2023).



Árvore não binária – Exemplo 2. Fonte: O autor (2023).

Agora, o que realmente diferencia as árvores de outras estruturas de dados é a maneira como elas organizam e acessam informações. Por exemplo: em uma lista ou vetor, a informação é **sequencial**. Para encontrar um item, muitas vezes precisamos percorrer a lista do início ao fim. Já as árvores têm uma abordagem **hierárquica**. Graças a isso, operações como busca, inserção e remoção podem ser **muito mais rápidas**, especialmente em árvores bem-balanceadas, em que os dados são distribuídos de forma mais ou menos igual em todos os "galhos". Esta eficiência torna as árvores instrumentos valiosos em muitas aplicações práticas, desde sistemas de arquivos em sistemas operacionais até bancos de dados e jogos. Falando nisso, veja só algumas aplicações comuns de árvores no mercado:

1. Se você tem interesse por desenvolvimento de *front-end*, o *Document Object Model* (DOM) é uma representação de árvore da estrutura de uma página da *web*. Cada elemento HTML é um nó na árvore, e os desenvolvedores frequentemente precisam navegar ou modificar essa estrutura. Como você pode imaginar, conhecer árvores pode lhe ajudar a manipular o DOM de um jeito mais eficiente.
 - a. Além disso, em *frameworks* de *sites* como o React, o "Virtual DOM" é uma árvore de nós que representa a página da *web*. O algoritmo de reconciliação, que determina como atualizar a página da *web* de maneira eficiente, é baseado em uma comparação de árvores.
2. Se você tem interesse por bancos de dados relacionais (lembra-se do SQL?), as árvores são frequentemente usadas para a indexação dos dados. Logo, é o uso delas que torna as operações de busca muito mais rápidas.

3. Geralmente, usamos estruturas de árvore para representar hierarquias de produtos ou categorias em sites de *e-commerce*.
4. Muitos jogos, especialmente os estratégicos, empregam árvores de busca para avaliar movimentos e tomar decisões no jogo.
5. Também é comum termos algoritmos de inteligência artificial que são árvores de decisão gigantescas (ou comitês de árvores de decisão), construídas de forma automatizada a partir de uma base de dados já existente.

Árvore binária em Java

Agora, como implementaríamos uma árvore binária em Java? A implementação é mais complexa em comparação com o que fizemos até agora. Então, vamos por partes. O primeiro passo é definirmos um nó. Veja que um nó de uma árvore sempre deve ter pelo menos três informações: o valor do próprio nó (que pode ser uma *string*, um número ou qualquer outro dado) e a identificação dos seus dois nós filhos: o que deve estar à esquerda e o que deve estar à direita. Portanto, vamos começar assim:

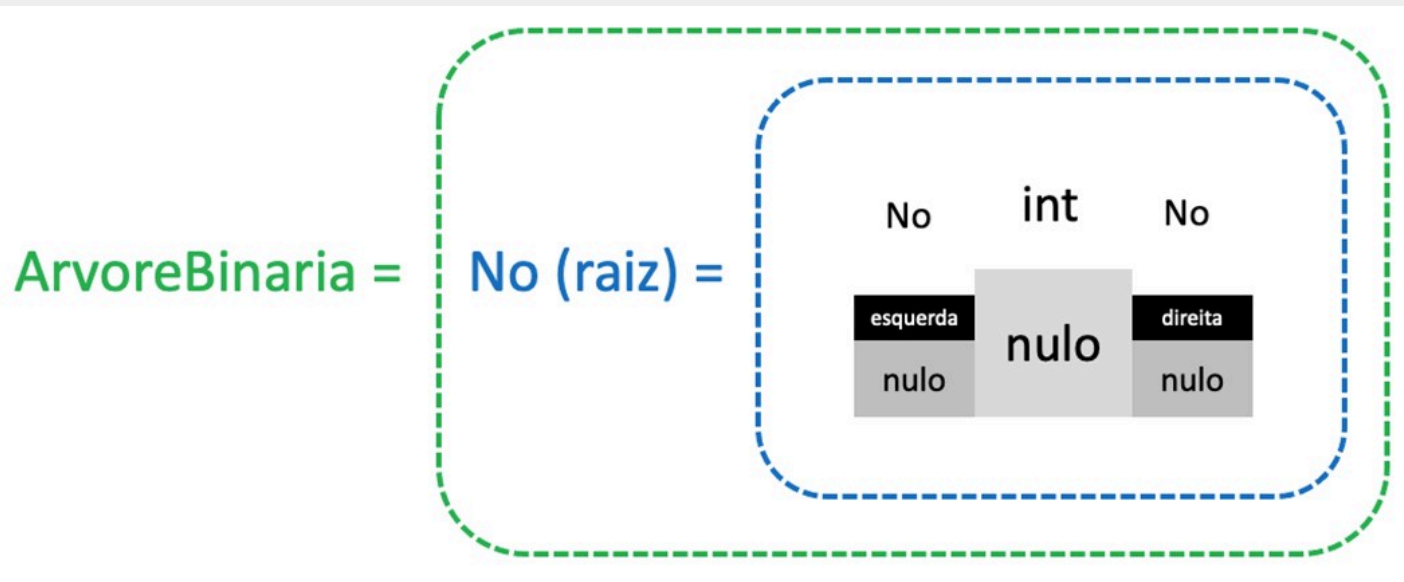
```
1 class No {  
2     int valor;  
3     No esquerda;  
4     No direita;  
5  
6     public No(int valor) {  
7         this.valor = valor;  
8         esquerda = null;  
9         direita = null;  
10    }  
11 }
```

Veja que definimos os nós de esquerda e direita como nulos: afinal, não sabemos ainda se este nó terá ou não algum filho. Agora, será o momento de definirmos uma classe para a nossa árvore. Começaremos com uma definição mais simples, contendo somente a funcionalidade de inclusão de nós e o construtor. Portanto, adicione isto ao seu código:

</>

```
1 class ArvoreBinaria {
2     No raiz;
3
4     public ArvoreBinaria() {
5         raiz = null;
6     }
7
8     // Inserção
9     public void inserir(int valor) {
10         raiz = inserirRecursivo(raiz, valor);
11     }
12
13     private No inserirRecursivo(No atual, int valor) {
14         if (atual == null) {
15             return new No(valor);
16         }
17
18         if (valor < atual.valor) {
19             atual.esquerda = inserirRecursivo(atual.esquerda, valor);
```

Aqui, estamos definindo uma árvore binária a qual obrigatoriamente possui pelo menos um nó, chamado de raiz. Esta raiz começará vazia (nula), e não terá nós filhos no início. Portanto, os nós da esquerda e direita também são nulos quando a árvore é criada. Veja a imagem a seguir, a qual ilustra isto:



Árvore binária vazia. Fonte: O autor (2023).

Ainda, dentro da classe **ArvoreBinaria**, temos a lógica para inserirmos os nós com o uso da recursividade.

Recursividade

Recursividade é um tema comum em programação para denotar as funções que chamam a si mesmas. Vamos entender recursividade em três passos simples? Apenas avance no nosso conteúdo depois de percorrer pelos três passos a seguir.

Passo 1: Comece lendo o passo 1.

Passo 2: Reflita por um momento sobre o que você leu dentro desta seção sobre recursividade.

Passo 3: Volte para o passo 1.

Se você chegou até aqui, significa que em algum momento, **entendeu** o que era recursividade após percorrer pelos três passos algumas vezes. No mundo da programação, temos uma piada clássica sobre recursividade: “**Para entender a recursividade, primeiro você precisa entender a recursividade**”.

Resumo: a recursividade é um método em que uma **função chama a si mesma** para resolver um problema, enquanto loops repetem um conjunto de instruções até que uma condição seja atendida.

Bom, era importante falarmos sobre recursividade primeiro porque o nosso código possui isto. Agora, vamos entender melhor o nosso código com a definição da **ArvoreBinaria**:

+ No raiz

É o topo da árvore. Como comentamos anteriormente, todas as inserções na árvore começam a partir da raiz.

+ `public void inserir(int valor)`

Este método é o ponto de partida para inserir um novo valor na árvore. Ele começa a operação a partir da raiz.

+ private No inserirRecursivo(No atual, int valor)

É aqui que a mágica (ou deveríamos dizer, a recursividade) acontece.

- **if (atual == null):** verifica se a posição atual está vazia. Se estiver, coloca um novo **No** (livro) lá.
- **if (valor < atual.valor):** se o valor que queremos inserir for menor que o valor do nó atual, então precisamos mover para a esquerda. E para isso, chamamos o próprio método **inserirRecursivo** novamente, mas desta vez com **atual.esquerda** como sendo o nó atual.
- **else if (valor > atual.valor):** se for maior, movemos para a direita. Novamente, chamamos o método recursivamente, mas agora com **atual.direita** como sendo o nó atual.
- **return atual:** ao final de cada chamada, retornamos o nó atual, que pode ser um nó recém-criado ou um nó existente.

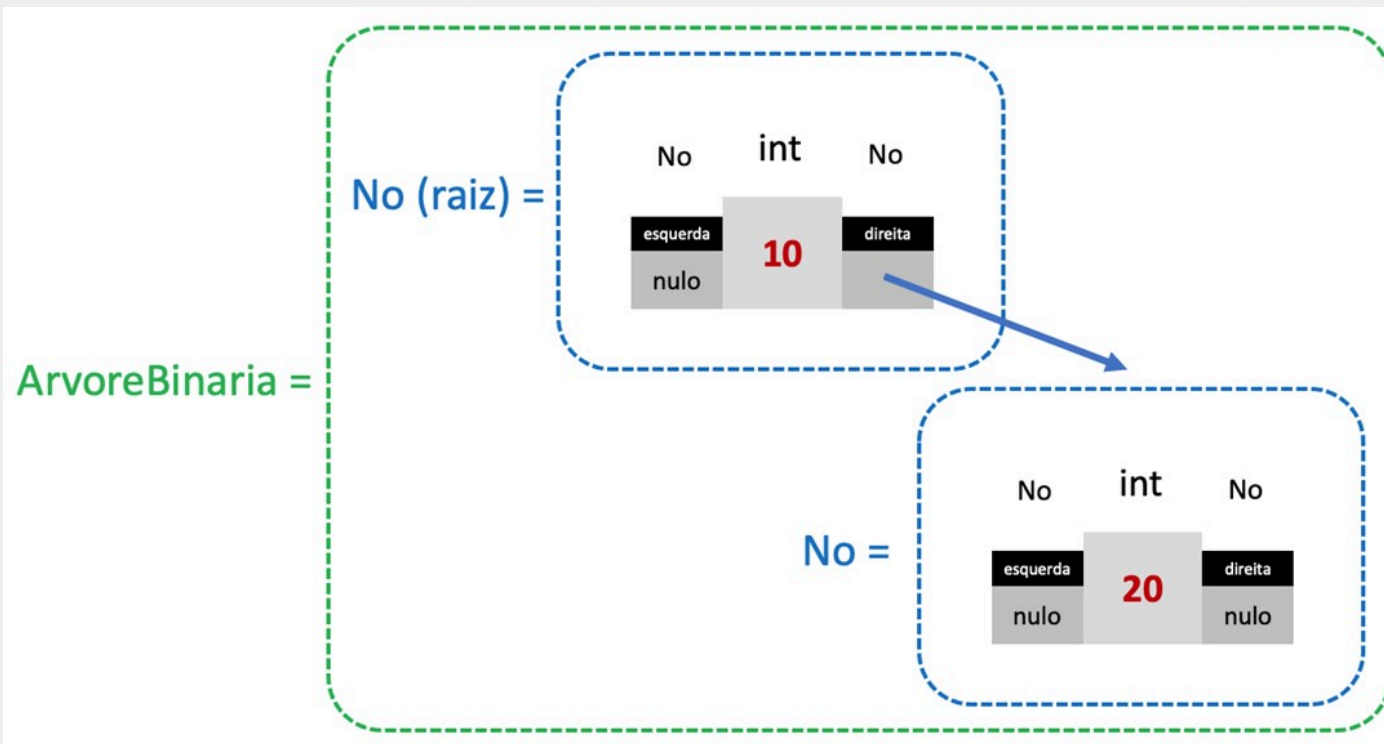
Vamos pensar em como funcionaria esta lógica? Vamos supor que queiramos criar a nossa primeira árvore e o primeiro elemento dela será 10. Neste caso, o nó **raiz** passa a ter o valor 10. Como não temos nós filhos (ainda), **esquerda** e **direita** permanecem nulos. Veja como fica na imagem a seguir:

ArvoreBinaria = No (raiz) =



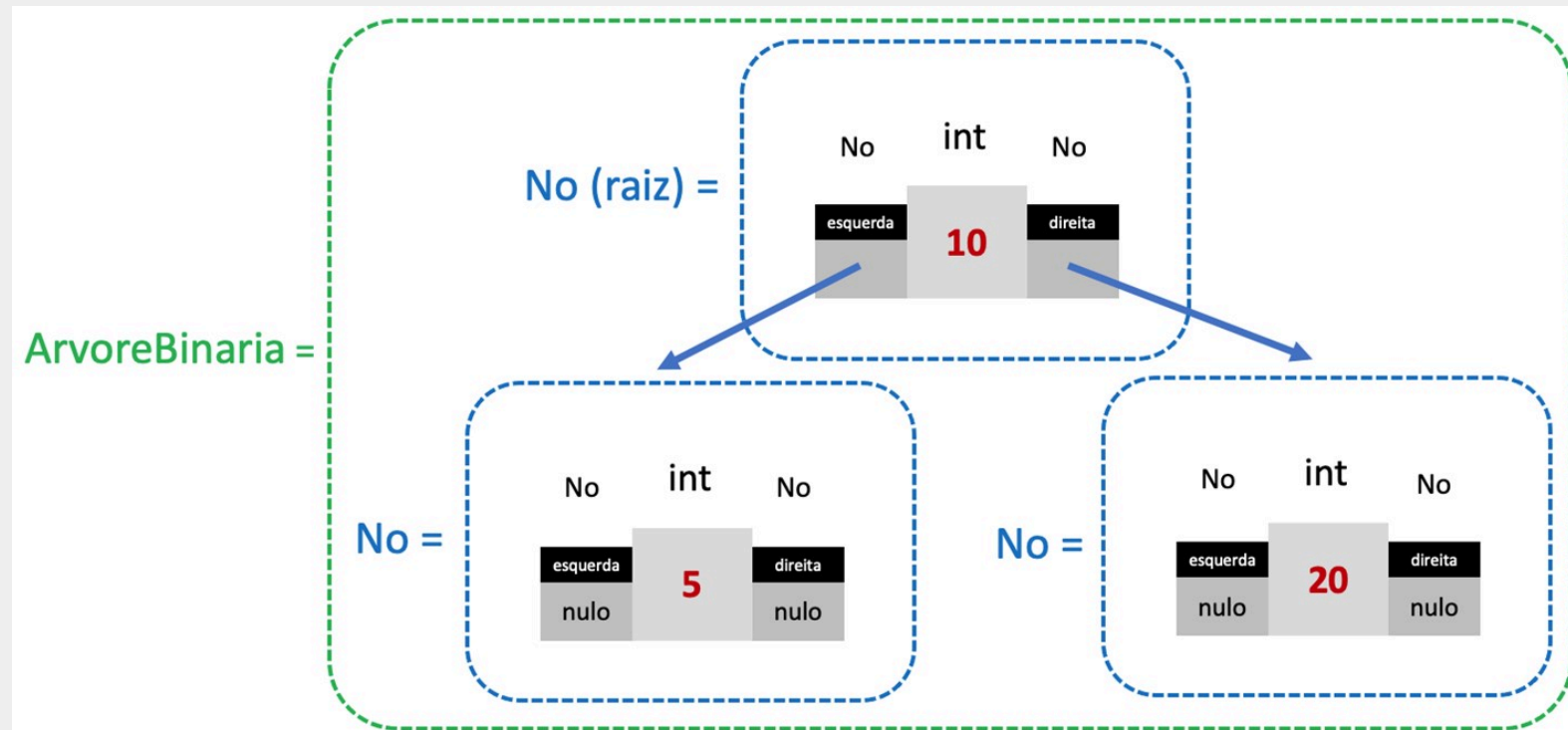
Árvore binária: inserção do valor 10. Fonte: O autor (2023).

Agora, e se inseríssemos outro valor, como **20**? Neste caso, criamos outro nó à direita da raiz, já que 20 é maior do que 10. Veja que esta lógica está presente dentro de **inserirRecursivo**. Veja que a referência do **No direita** da raiz passa a referenciar o nó com o valor 20.



Árvore binária: inserção do valor 20. Fonte: O autor (2023).

Continuando: e se inseríssemos um novo valor, como 5? Neste caso, criamos outro nó à esquerda da raiz, uma vez que 5 é menor do que 10. Veja que esta lógica está presente dentro de **inserirRecursivo**. Assim, a referência do **No esquerda** da raiz passa a referenciar o nó com o valor 5.



Árvore binária: inserção do valor 5. Fonte: O autor (2023).

Pesquisa (ou busca) de elementos em uma árvore

Veja que a lógica de inserção de elementos (isto é: valores menores sempre à esquerda e valores maiores sempre à direita) garante uma certa **ordenação na hora de pesquisarmos elementos**. Por isso, vamos implementar uma lógica de busca simples chamada de busca binária. Muito em breve, estudaremos em maior profundidade os métodos de pesquisa (ou busca) mais comuns. Dito isso, vamos implementar este método simples: ele será a base para criarmos a lógica para a remoção de elementos. Portanto, vamos modificar o código da classe **ArvoreBinaria** para incluir os métodos de busca. Então, o nosso código passa a ficar assim:

</>

```
1 class ArvoreBinaria {
2     No raiz;
3
4     public ArvoreBinaria() {
5         raiz = null;
6     }
7
8     // Inserção
9     public void inserir(int valor) {
10         raiz = inserirRecursivo(raiz, valor);
11     }
12
13     private No inserirRecursivo(No atual, int valor) {
14         if (atual == null) {
15             return new No(valor);
16         }
17
18         if (valor < atual.valor) {
19             atual.esquerda = inserirRecursivo(atual.esquerda, valor);
```

Vamos entender o que o nosso código faz dentro dos métodos **buscar** e **buscarRecursivo**:

+ Método **buscar(int valor)**

- Este é o método público de busca. Ele serve como ponto de entrada para a busca, começando pela raiz da árvore.
- Ele chama o método **buscarRecursivo(raiz, valor)** e retorna seu resultado.

+ Método buscarRecursivo(No atual, int valor)

- Este é o coração da busca, e ele opera de forma recursiva (lembra da recursividade, né?). Vamos dividi-lo em partes:
 - **if (atual == null) { return false; }**: esta é a **base da recursão**. Se o nó atual for nulo (significando que chegamos a uma "folha" da árvore ou além dela), a função retorna **false**, indicando que o valor não foi encontrado nesse caminho.
 - **if (valor == atual.valor) { return true; }**: esta é a **condição de sucesso**. Se o valor que estamos buscando for igual ao valor do nó atual, a função retorna **true**, indicando que o valor foi encontrado.
 - **return valor < atual.valor ? buscarRecursivo(atual.esquerda, valor) : buscarRecursivo(atual.direita, valor);**: esta é a **decisão recursiva**. Se o valor buscado for menor do que o valor no nó atual, o método chama a si mesmo para continuar a busca à esquerda da árvore. Se for maior, ele busca à direita. Esse é o aspecto recursivo, em que a função chama a si mesma com um subconjunto do problema original. Este código é um exemplo de **operador ternário**: é outra forma de escrever o código sem usar *if* e *else*, usando apenas uma linha.

Remoção de elementos em uma árvore

Podemos reaproveitar o código da pesquisa para incluirmos a remoção de elementos da árvore. Já adianto que este processo de remoção é um pouco complicado. Por isso, pensaremos em alguns exemplos antes de atuarmos no código.

Pensemos em uma analogia para ajudar na compreensão: imagine que você esteja organizando sua biblioteca de livros em prateleiras. Se você precisa retirar um livro específico (por exemplo, um livro que está bem no meio da prateleira), você não apenas remove o livro, mas também precisa ajustar os livros adjacentes para preencher a lacuna. Caso contrário, ficará um espaço vazio no meio da prateleira. Por outro lado, remover um livro da extremidade é mais fácil do que remover um do meio.

A mesma lógica aplica-se aqui: é mais fácil remover um nó sem filhos ou com um filho do que um nó com dois filhos. No último caso, você precisa “reorganizar” sua árvore, assim como reorganizaria seus livros.

Dito isso, vamos pensar em como seria com uma árvore. O passo a passo seria assim:

1. Se o nó de interesse tiver um valor maior ou igual:

a. Continue a busca no nó à direita (passo 1 a partir deste nó à direita).

2. Senão, se o nó de interesse tiver um valor menor:

a. Continue a busca no nó à esquerda (passo 1 a partir deste nó à esquerda).

3. Senão, se estamos no nó de interesse (os valores são iguais):

a. Se for um nó folha (isto é, sem filhos), simplesmente o remova.

b. Senão, se tiver apenas um filho à direita:

i. Se este nó estiver à direita de seu pai, substitua-o pelo seu filho à direita.

ii. Se estiver à esquerda de seu pai, mova o seu filho à direita para a esquerda do pai.

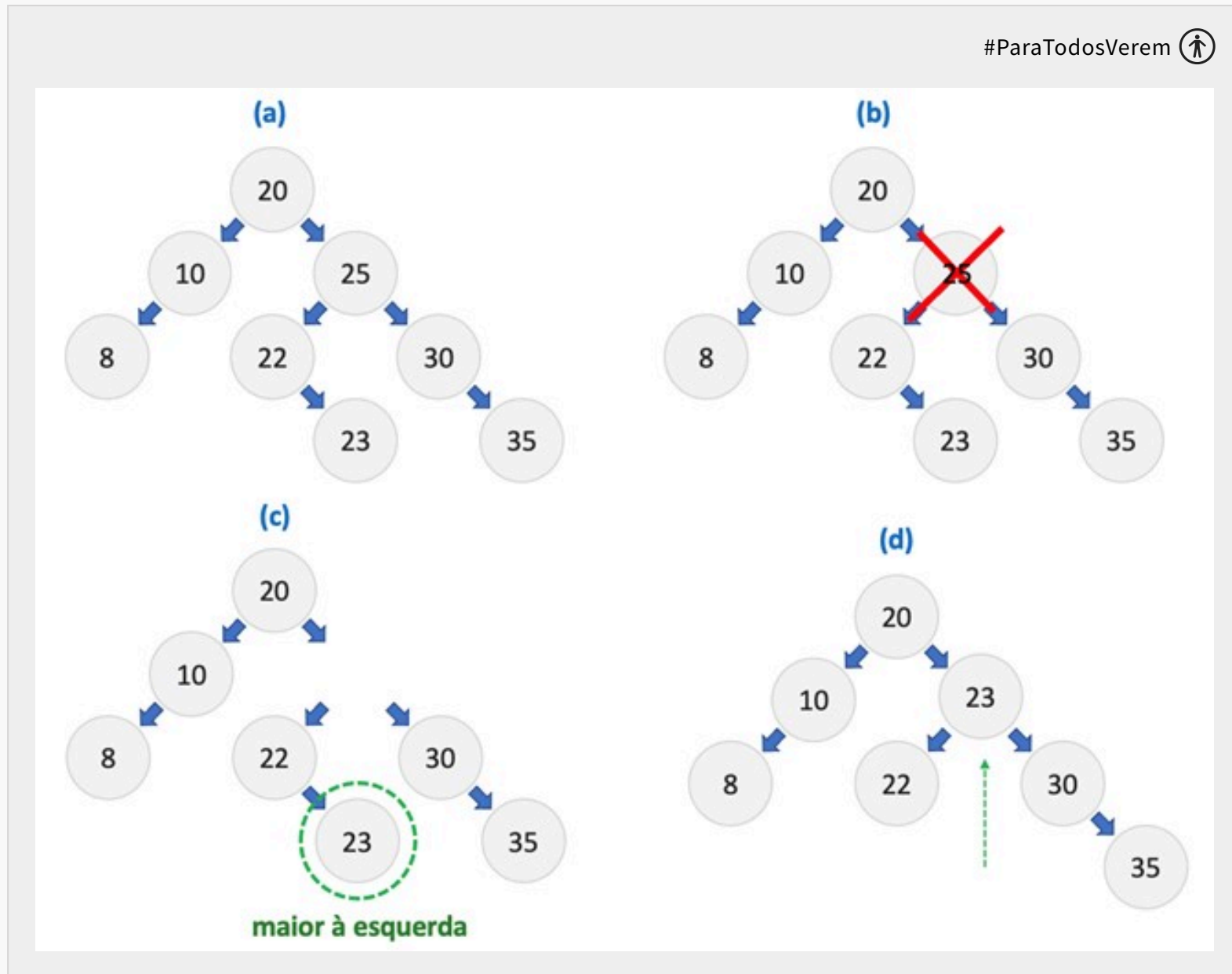
c. Senão, se tiver apenas um filho à esquerda:

i. Se este nó estiver à direita de seu pai, substitua-o pelo seu filho à esquerda.

ii. Se estiver à esquerda de seu pai, mova o seu filho à esquerda para a esquerda do pai.

d. Senão, se tiver dois filhos, substitua-o pelo maior valor da subárvore à esquerda.

Sei que este passo a passo fica difícil de ser entendido sem nenhum exemplo visual, não é? Pensemos em um exemplo complexo: a remoção de um nó com dois filhos. Veja a figura a seguir – o nosso desafio é o de remover o nó com o valor 25.



Veja que ao remover o valor 25 (passo b), fica um buraco vazio (passo c): aqui, precisamos de alguma forma manter a continuidade da nossa árvore. Portanto, precisaremos trocar aquele buraco vazio por algum outro nó que existe. Portanto, procuramos o nó com o maior valor da subárvore à esquerda do 25 (valores 22 e 23). Como 23 é o maior valor, ele assumirá o espaço do nó que foi removido (passo d), e a nossa árvore continuará funcionando.

Sendo assim, vejamos o código completo da classe **ArvoreBinaria**: agora, com três novas funções chamadas de **remove**, **removeRecurso** e **buscarMenorValor**. Veja como fica o código da classe com estas novas funções:

```
</>

1  class ArvoreBinaria {
2      No raiz;
3
4      public ArvoreBinaria() {
5          raiz = null;
6      }
7
8      // Inserção
9      public void inserir(int valor) {
10         raiz = inserirRecurso(raiz, valor);
11     }
12
13     private No inserirRecurso(No atual, int valor) {
14         if (atual == null) {
15             return new No(valor);
16         }
17
18         if (valor < atual.valor) {
19             atual.esquerda = inserirRecurso(atual.esquerda, valor);
```

+ Método `remover(int valor)`

- Este método serve como ponto de partida para a remoção. Ele invoca a função recursiva **`removerRecursivo`** e atribui o resultado de volta à raiz, assegurando que a árvore atualizada seja mantida.

+ Método `removerRecursivo(No atual, int valor)`

- Este método faz o trabalho pesado usando a recursão. Existem algumas tratativas diferentes dependendo do valor do nó:
- Se o nó atual (**`atual`**) for **`null`**, ele simplesmente retorna **`null`**.
 - Esta é a condição de parada da recursão e indica que o valor desejado não foi encontrado na árvore.
- Se o valor do nó atual for igual ao valor que queremos remover (**`valor == atual.valor`**):
 - **Sem filhos:** Se o nó não tiver filhos (nem à esquerda nem à direita), o método simplesmente retorna **`null`**, removendo assim o nó. Afinal, não precisaremos fazer mais nada além de só remover o nó.
 - **Com um filho:** Se o nó tiver apenas um filho (à esquerda ou à direita), ele retorna o filho não nulo. Dessa forma, esse filho **substituirá** o nó atual na árvore.
 - **Com dois filhos:** A abordagem aqui é encontrar o maior valor no lado esquerdo (subárvore à direita), substituir o valor do nó atual por esse maior valor e, em seguida, remover recursivamente esse maior valor da subárvore à esquerda. É **exatamente aquela explicação da figura anterior**, em que removemos o nó com o valor 25 e colocamos o nó com o valor 23 em seu lugar.

- Se o valor desejado for menor que o valor do nó atual (**valor < atual.valor**), a função chama a si mesma, mas agora analisando o filho à esquerda.
- Se for maior, a função chama a si mesma para o filho à direita.

+ Método buscarMaiorValor(No raiz)

- Este método é usado para encontrar o maior valor na subárvore fornecida. Ele faz isso navegando para a direita o máximo possível, porque na implementação desta nossa árvore o maior valor estará no nó mais à direita.

+ Método mostrar() e mostrarRecursivo()

- São dois métodos extras que criamos somente para mostrar no terminal a árvore.

Agora, o que acha de testarmos esta lógica? Vamos criar na classe Main aquela árvore que demonstramos na figura anterior. Vejamos:

```
</>
```

```
1 public class Main {
2     public static void main(String[] args) {
3         ArvoreBinaria arvore = new ArvoreBinaria();
4
5         // inserindo a raiz
6         arvore.inserir(20);
7
8         // inserindo alguns filhos da raiz
9         arvore.inserir(25);
10        arvore.inserir(10);
11
12        // inserindo mais alguns nós
13        arvore.inserir(8);
14        arvore.inserir(30);
15        arvore.inserir(22);
16
17        // finalizando com mais dois nós
18        arvore.inserir(35);
19        arvore.inserir(23);
```

Veja que criamos a árvore e fomos inserindo os nós. Conforme íamos inserindo os valores, a árvore automaticamente ia distribuindo os nós à direita e à esquerda dependendo do valor que era inserido. Também procuramos por dois valores que existiam (25) e não existiam (70) na árvore. Com isso, apareceram os valores *true* e *false* como resultado da busca, respectivamente.

Se você rodar o código, verá o resultado assim:

```
true
false
Árvore antes da remoção:
20
L----10
```

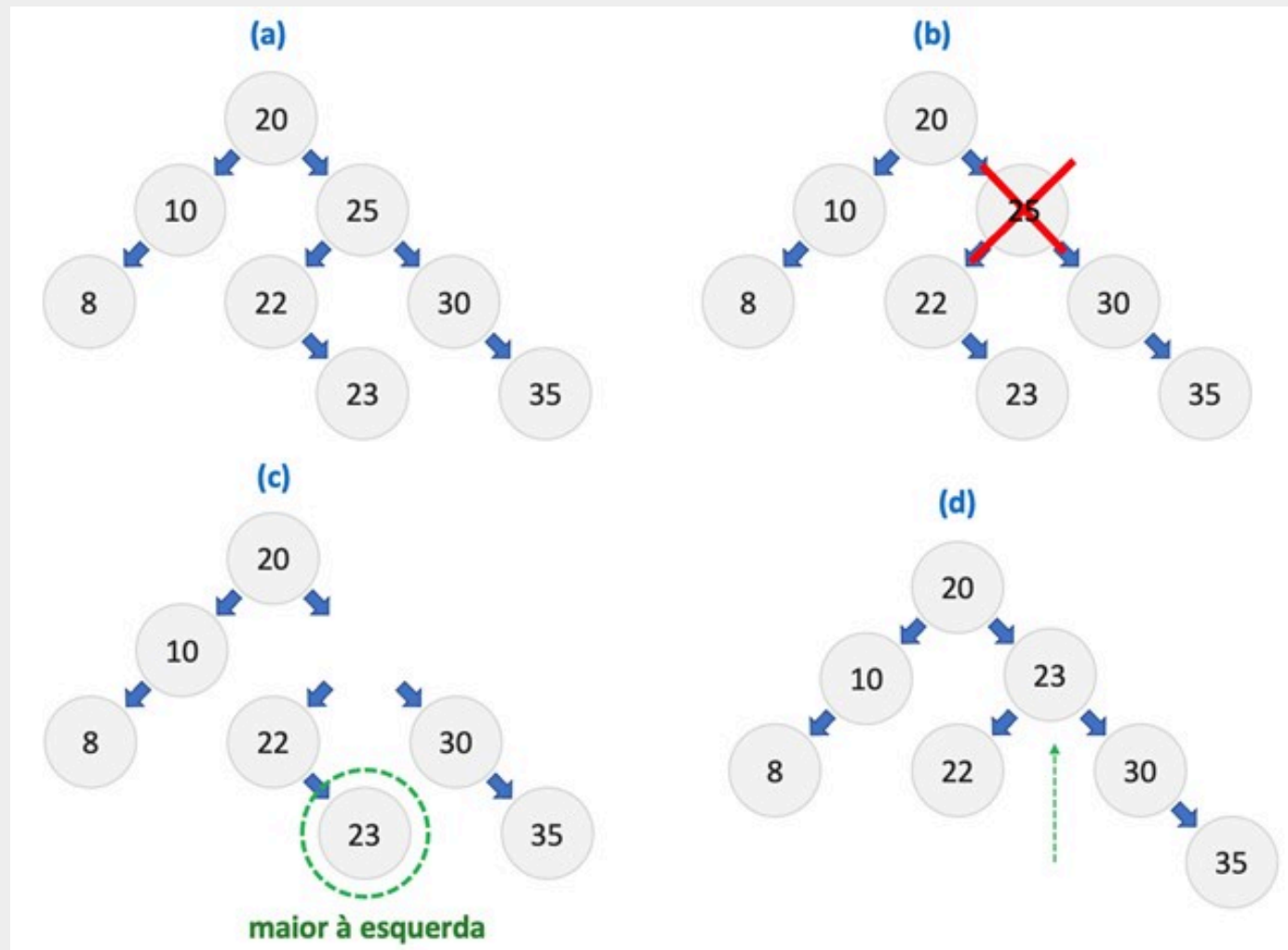
```
      L-----8
L-----25
      L-----22
          L-----23
      L-----30
          L-----35
```

false

Árvore depois da remoção:

```
20
L-----10
      L-----8
L-----23
      L-----22
          L-----30
              L-----35
```

Compare o antes e o depois com aquela árvore que vimos anteriormente – a incluímos novamente aqui a seguir para ficar mais fácil de se ver. É como se estivéssemos visualizando a árvore na horizontal:



Fonte: O autor (2023).

| Conclusão

Chegamos ao final deste conteúdo introdutório sobre árvores. Espero que tenha ficado claro o quão fundamental esta estrutura de dados é no universo da computação. Árvores não são apenas uma construção teórica, mas uma ferramenta real que usamos em várias aplicações no mercado.

Eu sei que o conteúdo sobre árvores é mais complexo em relação aos anteriores: não veja isso como um obstáculo, mas, sim, como um reflexo da sua flexibilidade e potência. Elas oferecem soluções para problemas que seriam incrivelmente desafiadores, senão impossíveis, de resolver com estruturas de dados mais simples. E, com essa complexidade, vem a necessidade de um estudo aprofundado e prática constante.

Independentemente da área de TI em que você atue, as chances são grandes de que, em algum momento da carreira, você se depare com desafios que demandam o uso dessa estrutura. Por isso, entender árvores o fará um profissional mais preparado, versátil e valorizado no mercado.

| Referências

TENENBAUM, A. M. **Estrutura de dados usando C**. São Paulo: Makron Books, 1995.



© PUCPR - Todos os direitos reservados.